



A survey of many-objective optimisation in search-based software engineering

Aurora Ramírez, José Raúl Romero*, Sebastián Ventura

Department of Computer Science and Numerical Analysis, University of Córdoba, Córdoba 14071, Spain

ARTICLE INFO

Article history:

Received 26 July 2018

Revised 18 November 2018

Accepted 17 December 2018

Available online 18 December 2018

Keywords:

Search-based software engineering

Many-objective optimisation

Multi-objective optimisation

Evolutionary algorithms

Literature survey

ABSTRACT

Search-based software engineering (SBSE) is changing the way traditional software engineering (SE) activities are carried out by reformulating them as optimisation problems. The natural evolution of SBSE is bringing new challenges, such as the need of a large number of objectives to formally represent the many decision criteria involved in the resolution of SE tasks. This suggests that SBSE is moving towards many-objective optimisation, an emerging area that provides advanced techniques to cope with high-dimensional optimisation problems. To analyse this phenomenon, this paper surveys relevant SBSE literature focused on the resolution of many-objective problems. From the gathered knowledge, current limitations regarding problem formulation, algorithm selection, experimental design and industrial applicability are discussed. Through the analysis of observed trends, this survey provides a historical perspective and future lines of research concerning the adoption of many-objective optimisation within SBSE.

© 2018 Elsevier Inc. All rights reserved.

1. Introduction

Software engineers daily make decisions for a variety of tasks, such as planning the best schedule to guarantee project completion or selecting the subset of requirements for the next release that satisfy stakeholders the most. In these situations, manually evaluating all possible alternatives is impractical, specially if many – often conflicting – decision criteria are involved. To support engineers, search-based software engineering proposes reformulating software engineering activities as optimisation problems to be automatically solved by search algorithms (Harman et al., 2012).

Particularly, a key element in SBSE is the definition of the fitness function, which allows numerically assessing the quality of solutions. It is usually based on software metrics, and serves to guide the search algorithm. For instance, code coverage can be used to find the minimum test suite, while the minimisation of the number of code smells can be used to guide a refactoring process. It is hard to imagine that complex SE activities could be simplified to the extend that only one metric is required. It is most likely that SE problems present several conflicting goals (Harman et al., 2012). However, it does not necessarily imply that formulating multiple objective functions is straightforward.

* Corresponding author.

E-mail addresses: aramirez@uco.es (A. Ramírez), jromero@uco.es (J.R. Romero), sventura@uco.es (S. Ventura).

Nevertheless, even two or three metrics could be insufficient to express all the decision criteria appearing in standard SE tasks, leading to the formulation of SBSE problems with a larger number of objectives. This implies that SE multi-objective problems are turning into *many-objective problems*, a term coined to refer to those problems with four or more objectives (Chand and Wagner, 2015). This new perspective represents an important challenge, since popular multi-objective algorithms present serious difficulties when a large number of objectives is considered. Opportunely, many-objective optimisation has become an active focus of research (Chand and Wagner, 2015), with a proliferation of specialised algorithms in the last years (Li et al., 2015a). The parallelism in the development of both fields, i.e. SBSE and many-objective optimisation, suggests a potential synergy that some SBSE authors are beginning to exploit (Hierons et al., 2016; Fleck et al., 2017; Panichella et al., 2018).

At this stage, the aim of this paper is to bring this emerging phenomenon to light as a way to encourage the SBSE community to further explore the benefits of many-objective optimisation. Following a methodological literature search and analysis (Kitchenham and Charters, 2007), this survey first digs into the origins of many-objective optimisation problems (MaOPs) within SBSE. After completing the search procedure, 72 papers were selected within the scope.

Although this is a significant number of publications that could suggest that substantial work has been done since the origin of SBSE, the reality is that SBSE is just beginning to discover the real potential of many-objective optimisation. Fig. 1 shows this upward

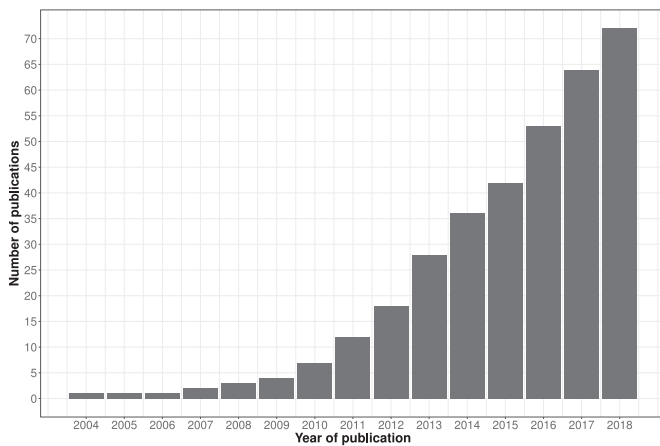


Fig. 1. Cumulative number of many-objective publications in SBSE.

trend by depicting the cumulative number of publications in SBSE addressing many-objective problems per year.¹ As can be seen, half of the contributions have been published after 2014.

This survey analyses for the first time how SBSE is progressively incorporating emerging ideas from many-objective optimisation. Our findings suggest that a variety of many-objective problems have already been defined in SBSE, but only few approaches actually apply many-objective optimisation techniques. Nevertheless, this observation seems to be changing, and recently published studies are exploring new algorithms and approaches to solve more complete problem formulations. Hence, there are still some gaps and challenges that could open interesting lines of future research.

The rest of the paper is organised as follows. Section 2 provides some background on SBSE and many-objective optimisation. Section 3 details the methodology followed for conducting the literature survey. Sections 4–7 cover the different phases of the software life cycle, proposed as optimization problems solved from the many-objective perspective. This phase categorisation follows the literature standards in SBSE (Harman et al., 2012). The analysis of the most relevant findings is explained in Section 8, while Section 9 discusses gaps and challenges. Finally, Section 10 concludes the paper.

2. Background

This section poses the theoretical foundations of the two topics tackled in this survey. Firstly, SBSE is introduced, and then the most relevant concepts and algorithms for many-objective optimisation are explained.

2.1. Search-based software engineering

The software engineering community has recognised the benefits of using automated techniques to support its increasingly complex processes. Particularly, SBSE has gained increasing attention since its appearance in 2001 (Harman and Jones, 2001). In short, SBSE proposes reformulating software engineering tasks as optimisation problems, so that efficient search algorithms like metaheuristics (Boussaïd et al., 2013) can be then applied to find a nearly optimal solution. Any SBSE approach requires the definition of two elements (Harman et al., 2012): a computational representation of the SE problem and a fitness function. The representation provides a machine-readable abstraction of the real-world problem, often encoded in the form of numerical or binary arrays, or

other data structures like trees. The fitness function is a means of numerically assessing the quality of candidate solutions, so that the algorithm can discern between better and worse alternatives. The fitness function is frequently based on software metrics as typically calculated by engineers, such as effort in project planning, cohesion in design-related tasks or code coverage in testing.

However, software engineering presents decision scenarios in which two or more criteria should be taken into account and a trade-off between them is expected. For this reason, SBSE promptly moved onto the application of multi-objective algorithms (Coello Coello et al., 2007), which can simultaneously optimise several objective functions. In this context, a survey reports more than 50 SBSE publications using multi-objective algorithms (Sayyad and Ammar, 2013). Another survey focuses on the adoption of quality indicators to assess the performance of this type of algorithms in SBSE (Wang et al., 2016). Related to this topic, a recent review presented some guidelines on how to select these quality indicators (Li et al., 2018a).

2.2. Many-objective optimisation

Many-objective optimisation arises as a natural extension of multi-objective optimisation. A solution x to a multi-objective problem (MOP) is represented by a number of decision variables $x_1, x_2, \dots, x_m$ and a number of constraints, $c_1, c_2, \dots, c_k$ (Coello Coello et al., 2007). Solving a MOP therefore consists in determining the values for all decision variables that minimise (or maximise) a number of objective functions, $f_1(x), f_2(x), \dots, f_n(x)$, while satisfying the constraints. The Pareto dominance establishes a comparison criterion between candidate solutions. A solution x dominates another solution y if x has equal or better objective values than y for $k - 1$ objectives, and is strictly better in one. The set of Pareto optimal solutions constitutes the Pareto set, and their projection into the objective space is called the Pareto front (PF). Quality indicators are used to measure the quality of the returned PF. Well-known examples are hypervolume (HV), generational distance or spread. These metrics quantify aspects like the distance to the optimal PF or the diversity among solutions.

Multi-objective evolutionary algorithms (MOEAs) have been widely applied to solve MOPs (Zhou et al., 2011). Based on the principles of natural evolution, MOEAs first create a population of random solutions, or individuals, that are then “evolved”. Iteratively, some individuals are selected to act as parents, from which new solutions are generated by applying crossover and mutation operators. Depending on their quality, i.e. objective values, these new solutions replace the existing ones. The most popular algorithms – also within SBSE – are NSGA-II and SPEA2 (Sayyad and Ammar, 2013). Other metaheuristics that imitate biological processes are Particle Swarm Optimisation (PSO) and Artificial Bee Colony (ABC) optimisation, among others (Boussaïd et al., 2013).

According to recent literature, many-objective optimisation problems require the definition of four or more objectives (Li et al., 2015a). A first mention to this type of problems dates back to the early 2000 s (Farina and Amato, 2002). Then, other authors began to study their particularities and to analyse how capable of addressing them were the existing MOEAs (Purshouse and Fleming, 2003; Fleming et al., 2005; Ishibuchi et al., 2008). Recently, MaOPs have attracted increasing attention and the following challenges were identified (López Jaimes and Coello Coello, 2015; Li, 2015): (a) the number of non-dominated solutions exponentially grows with the number of objectives, making it difficult to discriminate among them; (b) as a consequence, pareto-based algorithms excessively rely on the diversity preservation mechanism; (c) ensuring the balance between proximity to the PF and diversity becomes more difficult; (d) the effectiveness of crossover operators and density estimation techniques decreases, meaning that

¹ Up to the 30th of June of 2018.

Table 1
Families of many-objective algorithms.

Family	Description	Algorithms
Relaxed dominance-based	Relaxed forms of dominance are considered, so that better discrimination of non-dominated solutions is possible.	ϵ -MOEA (Laumanns et al., 2002), GrEA (Yang et al., 2013), MDMOEA (Zou et al., 2008), OMOPSO (Sierra and Coello Coello, 2005), SMPSO (Nebro et al., 2009)
Diversity-based	Special mechanisms to preserve diversity are included.	SPEA2+SDE (Li et al., 2014), VaEA (Xiang et al., 2017)
Aggregation-based	Objective values are aggregated in some steps of the search to compare solutions, which also allow establishing multiple search directions.	DBEA-Eps (Asafuddoula et al., 2013), MOEA/D (Zhang and Li, 2007), MOEA/DD (Li et al., 2015b), MSOPS (Hughes, 2003), NSGA-III (Deb and Jain, 2014), RVEA (Cheng et al., 2016)
Indicator-based	A quality indicator is used to evaluate the contribution of each solution to the quality of the approximated PF.	HypE (Bader and Zitzler, 2011), IBEA (Zitzler and Künzli, 2004), MOMBI (Gómez and Coello, 2013), MOMBI-II (Hernández Gómez and Coello Coello, 2015), SMS-EMOA (Beume et al., 2007)
Preference-based	User's preferences define the region of the PF to be searched.	R-NSGA-II (Deb and Sundar, 2006), PBEA (Thiele et al., 2009), P-MOET (Kalbousi et al., 2013), Two_Arch2 (Wang et al., 2015)
Dimensionality reduction	Redundant objectives are excluded from the optimisation process.	PCA-NSGA-II (Brockhoff and Zitzler, 2009), PCSEA (Singh et al., 2011)

new mechanisms have to be designed; (e) a greater number of solutions needs to be found to obtain good approximations of the PF; (f) more sensitivity of algorithm parameters is observed; (g) many-objective algorithms demand more time and memory resources; and (h) high-dimensional PFs require the use of specialised visualisation techniques (Walker et al., 2013).

For these reasons, algorithms specially well-suited for solving MaOPs have been proposed (Li et al., 2015a). Depending on the novel mechanisms designed to deal with the aforementioned challenges, many-objective algorithms are usually classified into different groups. Table 1 briefly describes current families, and provides a list of the most popular algorithms by category. In addition, notice that the need of comparing the performance of these many-objective algorithms has led to the definition of specialised indicators like PCI (Li et al., 2015c), pure diversity (Wang et al., 2017).

3. Methodology

This section describes the methodology followed for literature search and analysis, which is based on best practices (Kitchenham and Charters, 2007). The process involves both automatic and manual literature search, the selection of primary studies based on well-defined inclusion and exclusion criteria, and the corresponding data extraction. Each of these steps is detailed next. Additionally, different outcomes like the lists of papers and the sheet generated from data extraction can be found in the additional material.²

The search procedure combines automatic and manual search, as complementary mechanisms to mitigate the risk of missing relevant studies in the SBSE area. As for the automatic search, the following scientific databases were queried: ACM library, DBLP, IEEE Xplore, Web of Science, Scopus, SpringerLink and the SBSE repository.³ After a preliminary exploration of the content and keywords, Table 2 shows the two search strings agreed by all authors. Notice that “Search string #1” looks for publications whose title, abstract or keywords make reference to the general area of SBSE, whereas “Search string #2” is focused on specific phases of the software life cycle. The keywords selected for the latter were defined according to the ACM Computing Classification System⁴ and the IEEE Taxonomy⁵. Automatic search returned 237 distinct publications.

As a complement to the automatic search, all the authors conducted manual searches. The SBSE repository was scanned to retrieve those SBSE publications that do not explicitly mention the term “many-objective”, but actually define optimisation problems with more than three objectives. Additionally, a snowballing

Table 2
Search strings defined for literature search.

Search string #1	many-objective AND (optimization OR optimisation) AND (search-based OR search based) AND software engineering
Search string #2	many-objective AND (optimization OR optimisation) AND software AND (requirements OR architecture OR design OR development OR testing OR debugging OR verification OR maintenance OR evolution)

procedure was performed to check cross-references and detect manuscripts potentially omitted in previous searches. As a result, 143 new potential references emerged.

The next step consisted in filtering the list of publications to obtain a set of candidate papers. A first filter was based on the analysis of the title and abstract, after which a total of 122 distinct candidate papers were identified. Then, the exclusion and inclusion criteria shown in Table 3 were applied. At this stage, a decision was made by one author after reading abstract, introduction and conclusions. In case of conflict or doubt, the entire manuscript was scrutinised by the rest of authors. As a result of this process, 60 relevant studies were selected, plus 12 more studies reporting preliminary or duplicated results. These additional studies are counted for statistical purposes but excluded from the subsequent detailed review.

For each selected study, two authors extracted independently the following information: (1) meta-information; (2) search problem, including SE phase, problem definition, number of objectives and software metrics used as objectives; (3) search algorithm, including the type of metaheuristic, name and family of the main algorithm, its categorisation as newly proposed, adapted or adopted from the literature, comparison algorithms (if any), and software framework or library used for implementation; (4) experimental framework, covering the number and type of problem instances, quality indicators, visualisation techniques, statistical analysis and available material for reproducibility purposes. This information was gathered in spreadsheet forms along with the summary, future lines of research and additional comments for each contribution. A final summary sheet was compiled after solving any possible disagreement among authors.

4. MaOPs in project management and requirements

In the context of project management, SBSE has been mostly applied to predictive modelling, task scheduling and resource planning – often together with the so-called project scheduling –, all of them being critical for the success of the project (Ferrucci et al., 2014). All these activities require the trade-off among a large number of goals, that is, more than three objectives in this context. As detailed next in Sections 4.1 (for predictive modelling) and 4.2 (for

² <https://www.uco.es/grupos/kdis/sbse/many-objective>.

³ https://crestweb.cs.ucl.ac.uk/resources/sbse_repository/.

⁴ <https://www.acm.org/publications/class-2012>.

⁵ https://www.ieee.org/documents/taxonomy_v101.pdf.

Table 3
Exclusion and inclusion criteria.

Exclusion criteria	Inclusion criteria
Language is not English	The problem refers to a software engineering task
Full text is not accessible	A search algorithm is applied to solve the problem
Technical report or not peer-reviewed publication	The problem defines more than 3 objectives
Master/Ph.D thesis	Objective functions are not aggregated
Tool or competition paper with no contribution in algorithm or problem formulation	Theoretical proposals are allowed

project scheduling), authors have not explored the use of many-objective algorithms until very recently.

Similarly, requirements elicitation, selection and prioritisation are also decision-making scenarios in which engineers need not only to consider multiple criteria, but also to ensure diverse stakeholders' interests. Within this phase, search techniques have been mostly applied to select the optimal subset of requirements to be part of the next release, popularly known as the Next Release Problem (NRP) (Pitangueira et al., 2015). Section 4.3 summarises the only two studies that, to the best of our knowledge, have considered a many-objective formulation for the NRP, even though none of them apply many-objective algorithms.

4.1. Predictive modelling

A study on fault estimation of program modules is the pioneering work within both search-based project management and SBSE with many objectives (Khoshgoftaar et al., 2004). In this problem, a number of software modules are ranked according to a set of software metrics, obtaining the so-called module-order model (MOM). NSGA-II is adapted to the genetic programming (GP)⁶ principles to maximise MOM accuracy at four different threshold values, treated as independent objectives, while minimising the estimation model size. A later study (Sarro et al., 2012) analysed the influence of five accuracy metrics in effort estimation methods built by NSGA-II. It revealed that performance is greatly affected by the metric selected as objective, and single-objective GP can even outperform the five-objective approach.

More recently, two algorithms, originally conceived as general methods for any SBSE problem with a high evaluation cost, have been applied to model decisive factors of the software process. On the one hand, GALE uses PCA and active learning techniques to only evaluate the most informative solutions, thus returning a small number of good enough solutions (Krall et al., 2015). GALE, which can be catalogued as a preference-based approach, has demonstrated its superiority against NSGA-II and SPEA2 in solving a variety of problems, including the optimisation of COCOMO variables according to four objectives (risk, effort, defects and months). A faster yet competitive optimiser is SWAY (Chen et al., 2018), a divide-and-conquer sampling method that pretends to be considered as baseline algorithm within SBSE. In the experiments, SWAY reports similar or slightly better results than state-of-the-art algorithms when adjusting the COCOMO model.

4.2. Project scheduling

Project portfolio selection is a type of project scheduling problem consisting in choosing an optimal subset of projects and arranging them according to a planning horizon (Kremmel et al., 2011). Five decision factors are defined as objectives: the potential revenue of the portfolio, the strategic alignment of the chosen projects, the resource usage distribution, the involved risk and the synergy effect due to project interdependencies. A new multi-objective algorithm, named mPOEMS, is the selected algorithm to

address this problem. A case study shows its superiority with respect to NSGA-II and SPEA2. Moreover, the novelty of the approach lies in the ability of the algorithm to optimise objectives for both project selection and scheduling. Another planning task tries to determine the best values for initial team size and schedule estimates (Rodríguez et al., 2011). Simulation packages produce the project data, and then NSGA-II is run to optimise five objectives related to time, cost, productivity and staff skills (novice and experienced). The addition of these two latter objectives results in an increase of the number of solutions available to the engineer, who can also establish thresholds during a post-analysis phase.

Another recurrent problem within search-based project scheduling refers to the optimal allocation of staff to tasks. A recent work (Shen et al., 2016) adds robustness and stability factors to the original bi-objective formulation, based on minimising project cost and duration, with the aim of creating more realistic scenarios. ϵ -MOEA is combined with different rescheduling methods and dynamic optimisation techniques to find robust project schedules. In a different work, employees' satisfaction is also computed as an additional objective (Shen et al., 2018). Furthermore, the problem formulation is enriched with new project properties, such as motivation and learning capabilities of the team, to dynamically model how their skills improve over time. A specialised memetic algorithm is proposed using Two_Arch2 as a basis, the adaptation of the multi-objective Two-Archive algorithm for solving MaOPs. With these new objectives and decision variables, the proposed approaches offer the engineer the possibility to make better decisions based on human-based factors too.

4.3. Software requirements

A first work addressing a many-objective NRP (Finkelstein et al., 2009) looked for subsets of requirements that could be considered "fair" by all stakeholders. Fairness was expressed in terms of fulfilled requirements, taking their absolute number, their absolute value, the percentage of their value and their cost as objectives. Two multi-objective algorithms, namely NSGA-II and Two-Archive, were executed to demonstrate the suitability of the approach, both showing similar performance.

A different approach (Zhang et al., 2011) deals with each stakeholder's satisfaction as an independent objective, instead of computing them all as a single objective, which is the most common practice for the NRP. In this way, the possibility of incorporating a larger number of objectives is opened up. Solutions to synthetic problems with up to 50 objectives and two real-world case studies with four and five objectives are computed by NSGA-II, Two-Archive and random search. Results show that Two-Archive outperforms the rest of algorithms, specially as the number of objectives increases.

5. MaOPs in software analysis and design

Search-based software design is focused on the semi-automation of design tasks for a variety of paradigms, including object-oriented and service-oriented design (Räihä, 2010). The class responsibility assignment problem consists in assigning

⁶ GP is a type of evolutionary algorithm specially well-suited for evolving tree structures, a usual encoding for prediction models.

methods and attributes to a number of classes (Bowman et al., 2010), while trying to produce a highly cohesive and low-coupled object-oriented design. Three coupling and two cohesion metrics serve to guide SPEA2, showing that a many-objective formulation is more adequate than the optimisation of a weighted sum of all the metrics.

Software architecture optimisation (Aleti et al., 2013) is another design area in which the benefits of many-objective optimisation have been recently explored. In particular, eight algorithms – SPEA2, NSGA-II, MOEA/D, IBEA, HypE, ϵ -MOEA, GrEA and NSGA-III – of five different families have been adopted to discover component-based software architectures from early class diagrams (Ramírez et al., 2016). Combinations of up to nine design metrics, focused on measuring three non-functional quality criteria (modularity, reusability and analysability), are used to guide the search. The scalability study provides insights on the behaviour of each type of algorithm and how different metric combinations affect architectural solutions.

Search-based model driven engineering (MDE) (Boussaïd et al., 2017) is another field in which a many-objective optimisation problem has been recently formulated, more specifically, the modularisation of model transformations (Fleck et al., 2017). This process is similar to module clustering, since modularisation operations are applied to assign transformation rules and helpers to different modules. NSGA-III is the selected algorithm to solve the problem, for which four objectives were defined: cohesion, coupling, number of generated modules and module deviation size. In this case, NSGA-III outperforms other algorithms, including ϵ -MOEA, SPEA2 and some single-objective approaches, for six benchmarks and one industrial case study.

Nevertheless, in terms of the application of many-objective approaches, the most recurrent areas within the analysis and design tasks are those related to software product lines (SPL) and service composition. Therefore, the following sections describe in detail the specific problems addressed in these fields, respectively.

5.1. Software product lines

SPL optimisation stands out as the design task in which many-objective optimisation has been more widely explored, according to our literature search. In fact, the resolution of problems related to software product lines has attracted growing attention (Lopez-Herrejon et al., 2015) in the last years. A first work in the area (Colanzi and Vergilio, 2012) applies NSGA-II to improve the design of product line architectures represented as class diagrams. The authors analyse the role of five objectives, which measure stability, coupling, two metrics for cohesion and reusability, and then they conclude that metrics and search operators specifically conceived for SPLs require further research.

A more widely studied problem concerning SPL configuration is the so-called optimal feature selection. For a given feature model (FM), which specifies how features can be combined to derive valid products, optimal subsets of features need to be chosen according to criteria like user preferences, reusability or cost. The first many-objective formulation proposed was comprised of five objectives, namely correctness, richness, cost, features used before and known defects (Sayyad et al., 2013b). The authors point out that algorithms others than NSGA-II and SPEA2 should be taken into consideration in SBSE, specially for those problems with a large number of objectives. Therefore, they selected six Pareto-based algorithms (NSGA-II, ssNSGA-II, SPEA2, FastPGA, MOCeII and MOCHC) and one indicator-based approach (IBEA). Taking two FMs as input, their experiments show that IBEA provides the best performance for problems having between two and five objectives. The authors also highlight the importance of including correctness as an objective, since FMs are often highly constrained. In fact, they have

examined this issue further, concluding that injecting feasible solutions in the initial population is necessary to cope with this type of FMs (Sayyad et al., 2013a). The proposed enhancement allows IBEA to find valid products for FMs with more than 6000 features.

Several other researchers have continued studying the feature selection problem taking the works by Sayyad et al. as a basis. For instance, the performance of GIA, an exact algorithm, and IBEA, which has become a baseline approach for comparison, were compared (Olaechea et al., 2014). In this case, the number of objectives varies between two and seven, depending on the specific FM. The authors conclude that GIA is a suitable approach for small FMs, whereas IBEA with optimised settings performs better for large FMs. SATIBEA is an improved variant of IBEA founded on constraint satisfaction solvers (SAT) and specific operators that has proven to be more effective than IBEA for the original five-objective formulation (Henard et al., 2015). Due to the achieved improvements, SATIBEA was chosen as a comparison approach to test the performance of SWAY (see Section 4.1) in the context of SPL. Additionally, other authors have replaced SAT solvers by satisfiability modulo theory (SMT) solvers to deal with non-boolean constraints, and analysed its impact on population creation and mutation (Guo et al., 2017). Similarly, SAT solvers allow analysing the FM to remove mandatory features from the search (Tan et al., 2015), so that IBEA is able to produce more solutions satisfying the constraints. Later, this algorithm turned into a two-population-based algorithm, named IBED, that uses differential evolution to preserve diversity (Xue et al., 2016). In both studies, the algorithm is compared against the original version of IBEA, NSGA-II, ssNSGA-II and MOCeII.

Another method for many-objective feature selection is SIP (Shrink Prioritise), a general approach that gives priority to correctness over the rest of objectives (Hierons et al., 2016). Apart from the five-objective formulation stated by Sayyad et al. two more problem-specific formulations with eight objectives are proposed, including additional aspects in the decision process, such as the number of developers and changes or cost and memory usage. SIP is then integrated into six algorithms of four different families: NSGA-II, IBEA, SPEA2+SDE and three variants of MOEA/D. Experiments showed that SIP makes constraint satisfaction easier, though no algorithm was clearly superior. Recently, other paper suggested that previous works lack of solution diversity and are still computationally expensive (Xiang et al., 2018). To overcome this, they propose the use of additional SAT solvers to repair infeasible SPL configurations, as well as the application of a new algorithm, named VaEA, to promote particular configurations. SAT-VaEA, which could be catalogued as a diversity-based approach, is compared against SATIBEA and SIP (combined with NSGA-II and SPEA2+SDE) on an extensive collection of FMs, considering four, five and eight objectives. Experiments confirm that SATVaEA is an efficient method, being able to optimise FMs with more than 60,000 features in less than ten minutes. Moreover, it guarantees the return of more diverse solutions than its competitors, meaning that the engineer would have more different alternatives to choose among.

Still focused on SPL configuration, another problem refers to FM adaptation during product evolution, for which SATIBEA (Saber et al., 2018) was adapted. As a result, eSATIBEA takes the optimal solutions for a previous FM and modifies them to conform with an evolved FM, in which features and constraints have been added and/or removed. It computes the same five objective functions used in other state-of-the-art approaches, and concludes that taking the previous configuration as starting point significantly reduces optimisation time. However, it is not highly effective if the new FM greatly differs from the previous one.

The identification of business services for product lines can be formulated as a graph partition of the business process variability model (Khoshnevis and Shams, 2017). Part of the identification

process relies on running NSGA-II to find candidate services, so that the engineer can choose a preliminary solution to work with. The algorithm is guided by four service-oriented design metrics (cohesion, coupling, granularity and business entity convergence) and one SPL-specific design metric (commonality degree), though the approach is flexible in this regard. Here, the empirical study confirms that, compared to manually-provided solutions, the automated approach provides better reusability and variability, while preserving logical consistency.

5.2. Service-oriented design

Web service composition is a popular problem in service-oriented computation, frequently explored in SBSE (Canfora et al., 2005). Its goal is to find the optimal binding of candidate services for a number of tasks that orchestrate the composite service. Objectives often represent estimations of Quality of Service (QoS) properties, such as cost, latency and throughput. Similarly to other phases, even when the problem has been frequently defined in terms of more than three objectives, most approaches still rely on multi-objective algorithms.

A first work combined a multi-objective algorithm, MOGA, with fuzzy multi-attribute group, a decision-making technique (Liu et al., 2007). This latter technique is used to derive an overall quality score for Pareto optimal solutions based on the expert's opinion, providing some support to the decision process. The five objectives used for the evolutionary search are cost, time, availability, reliability and reputation. The same objectives were considered by the first truly many-objective approach to deal with the service composition problem (De Campos Jr. et al., 2010). The authors opted for two aggregation mechanisms – average (AR) and maximum (MR) ranking – that were embedded into NSGA-II. They also pointed out that these mechanisms are able to find balanced values for runtime, price and reputation, although more extreme values were returned for reliability and availability. This work also represents the first mention to many-objective optimisation in SBSE according to our literature search.

Later works have been focused on defining even more objectives and exploring other techniques, including exact search. On the one hand, the optimisation of throughput, latency and cost for three user categories (platinum, gold and silver), together with the overall cost, results in the most complex formulation of this problem regarding the number of objectives (Wada et al., 2012). Two specific algorithms can be found for this problem: E^3 -MOGA, which looks for equally distributed solutions; and $X-E^3$, which prioritises solutions with extreme objective values. Although these algorithms do not exhibit specific properties to deal with many-objective optimisation challenges, they provide better results than NSGA-II. The relationship between the execution time and the guarantee of optimal results has been studied (Trummer et al., 2014) too, for which an exact search algorithm (A-EXACT), an heuristic method (A-HEUR) and a mixed approach (A-FPTAS) were proposed. The latter, A-FPTAS, guarantees a bounded approximation error to the true PF in polynomial time. The three methods are also compared against NSGA-II, SMPSO and AbYSS for a five-objective formulation.

The application of NSGA-III and the use of real QoS values are common characteristics of the latest publications. Firstly, F-MOGP combines a search reduction technique with the execution of a GP-oriented NSGA-III (Yu et al., 2015). NSGA-II and SPEA2 are used as baseline algorithms for comparison. This study focuses on data intensive services, and defines latency, execution cost, availability and accuracy as objectives. Results show that this method is able to outperform state-of-the-art algorithms, while reducing computational cost due to a preliminary dependency analysis. More recently, two multi-objective (NSGA-II and SPEA2) and six many-objective algorithms (MOEA/D, ϵ -MOEA, GrEA, IBEA, HypE and

NSGA-II) have been compared (Ramírez et al., 2017) when looking for the optimisation of nine QoS properties (response time, availability, reliability, throughput, latency, successability, compliance, best practices and documentation). Experiments reported that some algorithms can return solutions with better QoS values for specific subsets of properties.

6. MaOPs in software testing

Testing is a predominant task in SBSE, where researchers have identified the need of formulating fitness functions beyond code coverage, specially those referred to non-functional properties (McMinn, 2011; Harman et al., 2015). Nevertheless, despite the maturity of the field of search-based software testing (SBST), it is not until 2013 that testing problems have been extended in their formulation.

Regression testing is one common scenario for the application of SBST methods. Notice that when the system under test experiences any change, test suite minimisation is crucial to reduce the cost of re-testing the whole system. Cost, together with other three different coverage metrics, are the objectives to select the optimal subset of test cases (Zheng et al., 2016). Two variants of MOEA/D are proposed in this work, showing their superiority when compared to NSGA-II. Furthermore, these variants can also find test suites with full coverage. The inclusion of mutation testing in the evaluation process leads to the definition of an additional objective, named mutation score (Wei et al., 2017). In this case, the performance of six algorithms, i.e. NSGA-II, IBEA, MOEA/D (three variants) and SPEA2+SDE, from four families is compared, NSGA-II and MOEA/D-PBI reporting the best results. It is worth noting that the definition of mutation score as a new objective allows reducing cost while improving fault detection.

Finding the optimal setting of noise injection techniques for testing concurrent systems is another example of a many-objective problem in SBST (Hrubá et al., 2014). After a correlation analysis of the metrics that could serve as objective functions, the following were selected: execution time, data races, atomicity violations and concurrent pairs. Here, the experimental study is focused on determining the best parameter set-up for NSGA-II, since it provided better results than SPEA and SPEA2 in a preliminary analysis. More recently, up to seven objectives have been defined for search-based test case selection at system level, i.e. black-box testing (Lachmann et al., 2017). Based on available metadata, they measure aspects like business relevance, execution cost and failure probability, among others. Experiments with different combinations of two to five objectives are carried out, using NSGA-II as search algorithm. Based on the results, the authors highlight the importance of including risk-based objectives and the influence of metric combinations.

The generation and prioritisation of test cases deserve special mention in the SBST field because of the relevant number of existing many-objective approaches being explored. The following sections make a thorough review of existing literature on these subjects.

6.1. Test case generation

A first study for test case generation defining a many-objective problem deals with autonomous agent testing (Kalboussi et al., 2013). Both the problem representation and objectives are specifically designed for the case study, a cleaner agent module. More specifically, each test case represents a tuple of coordinates in a room, whereas the seven computed objectives refer to aspects like power consumption and safety distance, among others. A preference-based algorithm, called r-NSGA-II, is applied to study the effect of choosing different reference points to guide the

search. Another initial problem for test suite generation consisted in finding the best sequence of transitions (test paths) for a given state extended finite state machine (Asoudeh and Labiche, 2013). The problem has been theoretically defined in terms of four objectives (feasibility, similarity, coverage and cost) and solved by a customised multi-objective genetic algorithm. However, the case study used for the experimentation had no infeasible test paths, so only three objectives were actually optimised.

Focusing on object-oriented systems, a first proposal (Panichella et al., 2015) defined each branch coverage as an independent objective, instead of the usual total coverage. This innovative formulation drastically raises the number of objectives to even more than 1000 for open source projects, all of them equally important. Therefore, the authors developed their own preference-based algorithm, called MOSA, inspired by NSGA-II. In a later work (Panichella et al., 2018), the same authors proposed DynaMOSA, an improved version that dynamically focuses on the uncovered testing targets according to the branch, statement and mutant coverage. According to the results, DynaMOSA can reach the same coverage than MOSA in less time. For the experimental comparison, input programs with up to 16,624 branches, i.e. objectives, were studied. The authors claim that this is the largest number of objectives ever considered within SBSE and that it also exceeds the typical benchmarks used in many-objective optimisation. This work finally suggests that standard many-objective algorithms might not be powerful enough for some SBSE problems, so specialised methods could be required instead. In a different paper, MOSA had been extended to include coupling and cohesion as secondary criteria within its selection mechanism (Palomba et al., 2016). This variant, referred as quality-based MOSA, is able to generate shorter test cases with even more quality than MOSA, while increasing the overall code coverage.

Other studies have included MOSA for comparative purposes. MIO (Arcuri, 2018b), a many-objective evolutionary algorithm to automatically remove, add or modify test cases, is founded on specific properties of the testing targets, such as the presence of a large number of branches or possible interdependencies. Experiments show that MIO provides better overall performance than MOSA and the default search in EvoSuite for problems between 1 and 100 objectives. Nevertheless, the author suggests that the particularities of the system being tested could influence the behaviour of the algorithm. Both MOSA and DynaMOSA were also considered in a SBST comparative study whose aim is to prove the suitability of different variants of simple evolutionary algorithms and their optimal parametrisation (Campos et al., 2017). The number of objectives varies between one to eight for these algorithms. The authors conclude that many-objective algorithms outperform other approaches for branch coverage, but MOSA is less competitive for overall coverage. Finally, a replicated experiment comparing MOSA and LIPS, a single-objective algorithm for SBST (Panichella et al., 2017), shows the superiority of MOSA when a larger number of more complex input programs are considered.

6.2. Test case prioritisation

Other recurrent testing tasks within SBST are related to test case prioritisation. A first problem consists in ordering black-box test cases from state machines, so that those test cases having higher defect detection capabilities are executed first (Briand et al., 2013). The selected algorithm, SPEA2, seeks for maximising data flow cumulative coverage rates at a specified cost and budget, while maximising overall coverage and minimising overall test cost. Experiments are conducted for a manually-produced case study, and no comparison against other algorithms is presented.

MOCATO is a generic approach than can be combined with any multi-objective algorithm to determine the optimal order to inte-

grate and test program units in both object-oriented and aspect-oriented contexts (Assunção et al., 2014). The ultimate goal is to reduce the number of required stubs, which can be achieved by minimising four coupling measures: number of attributes, number of operations, number of distinct return types and number of distinct parameter types. The performance of the algorithm PAES is compared against NSGA-II and SPEA2 for formulations with two and four objectives. The same algorithms and objectives are revisited in a later work (Assunção et al., 2013), which is focused on examining whether the order in which aspects and classes are tested can influence the search. Furthermore, the authors have proposed a hyperheuristic algorithm inspired in NSGA-II, called HITO, that automatically selects the genetic operators to be applied in each iteration (Guizzo et al., 2017). In this case, the comparison algorithms are NSGA-II and MOEA/D.

From an overall perspective, when four objectives are considered, differences among algorithms become more evident. However, for some systems under test, the number of objectives does not influence. Actually, even though non-conflicting objectives are sometimes used, the authors argue that their optimisation is still convenient for returning solutions including some preferences like a greater variety of return or parameter types or specific code constructs.

7. MaOPs in software maintenance

Maintenance and evolution tasks refer to those decisions aimed at preserving or improving system quality once it is in operation (Bavota et al., 2014). Many-objective optimisation has been explored in the three main maintenance topics within SBSE: modularisation, software correction and refactoring.

7.1. Software modularisation

Software (re)modularisation looks for identifying an optimal partition of the graph that represents the dependencies among software artefacts, e.g. modules or packages. It is worth noticing that modularisation is also useful to understand the system structure (Räihä, 2010), and this problem may adopt similar optimisation procedures and metrics than design tasks. A frequently used metric in SBSE is MQ (Modularisation Quality), which looks for a trade-off between high cohesion and low coupling. In fact, MQ is part of the first two many-objective formulations, referred as MCA (Maximising Cluster Approach) and ECA (Equal-size Cluster Approach), comprising five objectives each (Praditwong et al., 2011). On the one hand, the objectives in MCA are maximising MQ, the number of clusters and the sum of intra-edges of all clusters, while minimising the sum of inter-edges and the number of isolated clusters. On the other hand, ECA replaces the latter objective by the difference between the maximum and minimum cluster size. Both groups of objectives are embedded into Two-Archive, ECA showing a better overall performance.

The computational cost of the aforementioned approach led to explore new formulations and techniques. Based on ECA, two new formulations, one discarding MQ and another one replacing it by EVM (Evaluation Metric Function) (Barros, 2012) were proposed. Experiments with NSGA-II demonstrate that the former formulation achieves better results while reducing execution time. Both ECA and MCA are then analysed (Amarjeet and Chhabra, 2014a). Their sensitivity study focused on the impact of MQ on the performance of NSGA-II, revealing that MCA is less sensitive to changes for this metric. In addition, they proposed a slight variation of the modularisation problem, in which core components of the modules need to be preserved (Amarjeet and Chhabra, 2014b). In both cases, these core components were identified before running the algorithm, NSGA-II, so they maintained the same formulation.

Two different approaches make use of more advanced techniques to deal with module clustering. On the one hand, an interactive approach allows the developer to give his/her opinion about the solution with highest MQ under the MCA formulation (Bavota et al., 2012). Two interaction mechanisms are proposed, one in which the human expert can specify whether two randomly picked modules should stay together, and another one in which the decision is whether an isolated module should be integrated in a different cluster. As a result, the developer is provided with more meaningful solutions than those produced by the non-interactive variants. On the other hand, a hyperheuristic approach has been applied to the modularisation problem with the aim of automatically determining the best combination of selection, crossover and mutation operators (Kumari and Srinivas, 2016). Compared to Two-Archive and NSGA-II, this approach returns better values for the three so-considered most important objectives (MQ, intra-edges and inter-edges), in both MCA and ECA formulations, in a shorter time. The hyperheuristic model was embedded into a graphic tool and was validated with a real case study.

Other recent modularisation studies are focused on the definition of new objectives. On the one hand, a first proposal uses structural, semantic and historical information to define seven completely new objectives (Mkaouer et al., 2015). A truly many-objective algorithm, NSGA-III, is applied for the first time in this work, which is also characterised by allowing more types of modularisation operations to change the package structure. The comparison is performed against MOEA/D, IBEA, some previous works based on NSGA-II and other single-objective algorithms. On the other hand, two minimisation objectives, namely cluster cyclic dependencies and average shorter path between clusters, are added to MCA and ECA (Amarjeet and Chhabra, 2018). Due to this extension, they design a many-objective approach based on ABC paradigm, fuzzy-Pareto dominance and the use of two external archives, which is then compared against Two_Arch2, NSGA-III, MOEA/D and IBEA. According to our literature search, this is the first and only work proposing a non-evolutionary approach to address a many-objective software engineering problem.

7.2. Software correction

A second area of interest in search-based software maintenance refers to automated software correction. A first proposal, called CASC system (Wilkerson and Tauritz, 2012), is intended to generate test cases in order to automatically detect and fix bugs. It combines NSGA-II with GP and maintains two populations to evolve code and test cases separately. The distance to the original source code is used to evaluate code similarity, while problem-specific objectives are considered for test case generation. Experiments test two and four objectives against two small programs. The results seem to indicate that CASC tends to optimise problem-specific objectives first, meaning that similarity could be better used as a secondary criterion.

Also related to bug fixing, the automated assignment of bugs to developers has been recently studied (Khalil et al., 2017). Two different formulations with three and four objectives are proposed here, measuring the number of fixed bugs (total, severe and high-priority) and the time left to fix the remaining bugs. Having used NSGA-II to find optimal assignments, results show that adding the time as an objective is not detrimental to the optimisation of the rest of objectives, while promoting the generation of more solutions.

7.3. Refactoring

Much of the recent work in search-based software maintenance is focused on finding optimal sequences of refactoring operations

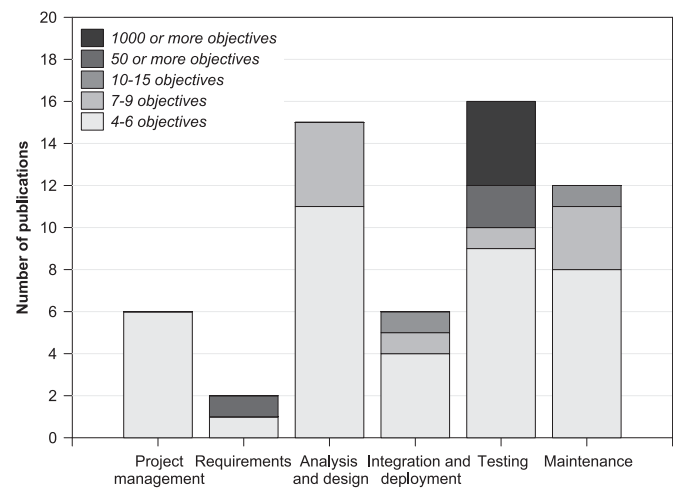


Fig. 2. Maximum number of objectives defined in each SE phase.

to remove bad smells and improve code quality (Mariani and Vergilio, 2017). The first work defining refactoring as a many-objective problem employs up to 15 software metrics to guide NSGA-III (Mkaouer et al., 2014), including those from the CK metric suite (Chidamber and Kemerer, 1994). The algorithms used for comparison are IBEA, MOEA/D and NSGA-II, as well as single-objective approaches. Experimental outcomes reveal that increasing the number of objectives could be beneficial to cover a wider spectrum of code smells, since each of them has an impact on diverse metrics. However, it slightly increases execution time too.

The same authors have reformulated the problem so that its objectives become the six external quality attributes defined in the QMOOD suite (Bansiya and Davis, 2002), together with the number of code changes and a design coherence preservation measure (Mkaouer et al., 2016). Additional many-objective algorithms, such as GrEA and DBEA-Eps, are included for comparative purposes. In this case, the authors highlighted that reusability, understandability and extendibility were the most improved characteristics. In contrast, those metrics related to functional aspects had shown less improvement. Coherently, refactoring is not intended to change the system behaviour. Finally, a theoretical proposal discusses the suitability of dimensionality reduction techniques in search-based refactoring (Dea, 2016). More specifically, the author suggests the use of PCA-NSGA-II, which applies principal component analysis (PCA) to select the objectives.

8. Discussion of findings

Previous sections have revealed a strong presence of many-objective optimisation problems in SBSE. Next, the most relevant findings from the surveyed proposals are discussed according to four dimensions: problem definition, algorithm selection, experimental validation and practical implications.

8.1. Problem definition

In relation to the definition of the search problems, it is worth highlighting how problem formulations with six or less objectives are predominant in SBSE so far. This fact is illustrated in Fig. 2, which shows the frequency of the number of objectives (in different intervals) considered in each phase. In the figure, only the problem formulation with the larger number of objectives is shown for those manuscripts containing more than one definition. General approaches (Shen et al., 2016; Chen et al., 2018) applied to multiple domains, as well as a theoretical study (Dea, 2016) that does not indicate the objectives being optimised, were excluded too. We speculate that such a relatively low number of objectives



Fig. 3. Timeline of many-objective problems in SBSE.

could suggest that there are still some limitations regarding the objective definition, probably because of the difficulty of mathematically formulating some subjective criteria applied by software engineers or adapting software metrics to a specific problem.

Many-objective problems with a large number of software metrics serving as objectives appear more frequently in three phases of the software life cycle: analysis and design, testing and maintenance. This is notorious considering that first publications in these fields appeared in 2010, 2013 and 2011, respectively (see Fig. 3). Even in these areas, the maximum number of objectives being optimised makes a difference. Design-related tasks have been defined in terms of nine objectives at most, while some problem definition in maintenance has reached up to 15 objectives. Finally, some cases in testing calculate an extremely large number of objectives (>50).

Nearly half of the analysed publications present various problem formulations with the aim of analysing the impact of a progressive increase in the number of objectives (Sayyad et al., 2013b; Mkaouer et al., 2015), the role of specific metric combinations (Praditwong et al., 2011) or both factors simultaneously (Ramírez et al., 2016; Lachmann et al., 2017). In addition, some studies for SPL optimisation and SBST define different number of objectives depending on the specific case study (Hierons et al., 2016; Panichella et al., 2018). It is also interesting to note that some studies reflect opposite opinions regarding the benefits of increasing the number of objectives. For instance, adding objectives has demonstrated to generate harder test cases (Kalboussi et al., 2013) and more variety of bug fixing assignments without demoting overall quality (Khalil et al., 2017). In contrast, effort estimation (Sarro et al., 2012) and software modularisation (Barros, 2012) seem to work better with fewer objectives. These cases could sug-

gest that including new objectives is not always beneficial and may require a previous, in-depth analysis.

Contrary to what would be expected, the conflict between objectives has been analysed in just a few works in the area, and not until very recently. Studying this correlation would enable the selection of the subset of truly conflicting objectives (Hrubá et al., 2014). It could be also studied during the search procedure by using dimensionality reduction algorithms (Dea, 2016). This point becomes a relevant aspect for engineers as well, insofar as they should be aware of the need of establishing trade-offs between measures (objectives) when choosing their solution. Nevertheless, it is not a trivial matter since it forces the software engineer to deeply know the nature of the objectives. Sometimes similar software properties appear in each phase, e.g. coverage criteria for testing, coupling and cohesion for design and maintenance tasks or QoS estimations for service composition. However, measures are not always consensual, and the lack of agreement regarding their effect on the software quality makes the objective definition a tough task. Besides, the use of customised objectives, which are often dependent on how the problem is encoded or the case study being solved, clearly hampers their applicability to other domains. These issues are commonly acknowledged within SBSE, but become more challenging when dealing with many criteria.

8.2. Algorithm selection

Many-objective problems are gaining interest within the SBSE community. However, it is not reflected in a broad adoption of many-objective algorithms. Table 4 lists the algorithms currently applied in the area, either proposed as main method or considered for comparative purposes. For those publications presenting

Table 4
Search algorithms applied in SBSE.

	Algorithm	Main method	Comparison
Pareto-based	AbYSS	–	1
	Custom. EA (*)	2	–
	dCOEA	–	1
	E ³ -MOGA (*)	1	–
	FastPGA	1	–
	HITO (*)	1	–
	MHypGA (*)	1	–
	MOCcell	1	2
	MOCHC	1	–
	MOGA	1	–
	NSGA-II	21	19
	mPOEMS	1	–
	PAES	2	–
	R/IC-IMGA (*)	1	–
	SPEA	–	1
	SPEA2	6	6
	ssNSGA-II	1	2
	Two-Archive	3	1
	X-E ³ (*)	1	–
Rel. Dom.	ϵ -MOEA	2	2
	d ϵ -MOEA (*)	1	–
	GrEA	2	1
	SMPSO	–	1
Diver.	SPEA2+SDE	2	1
	SATVaEA (*)	1	–
Aggregation	VaEA	–	1
	AR/MR	1	–
	DBEA-Eps	–	1
	F-MOGP (*)	1	–
	MOEA/D	5	5
	MOEA/DD	–	1
Indicator	MOEA/D-DE	–	1
	NSGA-III	6	3
	HypE	2	–
	eSATIBEA (*)	1	–
	FG-IBEA (*)	1	–
	IBEA	6	8
Preferences	IBED (*)	1	–
	SATIBEA (*)	1	1
	SMTIBEA (*)	1	–
	DynaMOSA (*)	2	–
	GALE (*)	1	–
	MIO (*)	1	–
	MOTAMAQ (*)	1	–
	MOSA (*)	3	3
	QB-MOSA (*)	1	–
	R-NSGA-II	1	–
DR	Two_Arch2	–	1
	PCA-NSGA-II	1	–
Exact	A-EXACT (*)	1	–
	GIA	1	–
Not classif.	A-HEUR (*)	1	–
	A-FPTAS (*)	1	–
	FP-ABC (*)	1	–
	SWAY (*)	1	–
	Other	–	20

a comparative study where no algorithm is more relevant than other, they all are considered as main methods. It is noticeable that NSGA-II is by far the most popular algorithm. We speculate that the reason could be twofold: the lack of specific many-objective algorithms in earlier stages of SBSE, and its good performance for the resolution of SE problems with two or three objectives. Notice that many-objective approaches were not applied until 2010, not even until 2014 for problems with more than five objectives. There seems to be a turning point in 2013. Since then, specific algorithms like IBEA and NSGA-III have become references of their respective families.

A noticeable fact is that authors have started to develop their own many-objective algorithms (marked with an asterisk in Table 4). These cases are often motivated by the authors. For instance, the high number of branches (objectives) and the need of

focusing on those difficult to cover is a recurrent reason in SBST. In SPL, the importance of returning valid products motivates the inclusion of specific constraint handling mechanisms. In general, the proposed algorithms either adapt state-of-the-art approaches like IBEA to the particularities of the problem under study, or propose new techniques with advanced capabilities like machine learning, e.g. GALE, hyperheuristics, e.g. HITO and MHypGA, or interactivity, e.g. R/IC-IMGA. These contributions help to stimulate the development of SBSE in pursuit of more effective search techniques. This is the case of MOSA, which has been rapidly adopted by the SBST community as a reference algorithm.

The rapid growth of many-objective optimisation in the last years makes the selection even harder. Opportunely, research efforts are now directed towards determining whether some algorithms are more suitable to particular many-objective problems. Recent experimental studies have based their findings on well-known properties of the Pareto front, the number of objectives or the conflict among them (Li et al., 2013; Li et al., 2018b; Li et al., 2018c). In this sense, from an evolutionary perspective, aggregation-based approaches perform well in problems with local fronts, but exhibit low performance in problems with disconnected and highly-dimensional fronts. Those algorithms based on a relaxed dominance principle usually obtain a uniform distribution of Pareto optimal solutions. Indicator-based algorithms seem to be oriented towards the solutions of the Pareto front that are favoured by the chosen indicator. For instance, when the hypervolume guides the search, the algorithm tends to focus on boundary solutions, i.e. those that present extreme values for some objective.

A special mention deserves the selection of a particular algorithm in SBSE, which does not seem to be motivated by these findings. This could be explained by the fact that most of these studies assume two factors that might not hold for SBSE: (1) most SE optimisation problems are not continuous, but combinatorial; and (2) their mathematical properties are unknown yet. As the theoretical basis in many-objective SBSE becomes more solid and general comparative studies provide further insights on real-world problems, the SBSE community would greatly benefit from their guidelines to make a better choice.

We speculate that the availability of public implementations also affects the selection of certain algorithms. Multi-objective algorithms can be easily found in metaheuristic optimisation frameworks and libraries, some of which are making important efforts to include many-objective algorithms. jMetal (Durillo and Nebro, 2011) is the most popular framework in SBSE, being used in 20 out of the 34 publications that provide implementation details. Other general-purpose frameworks used are JCLEC (Ventura et al., 2008), ECJ (Luke, 2017), DEAP (Fortin et al., 2012) and Matlab Optimisation Toolbox (MathWorks, 2018). EvoSuite (Fraser and Arcuri, 2011) and EvoMaster (Arcuri, 2018a) are the reference tools for SBST.

8.3. Experimental validation

The random nature of metaheuristic algorithms makes necessary the execution of multiple runs and the subsequent application of statistical tests to compare their performance (Arcuri and Briand, 2014). This is a good practice in SBSE followed by 70% of the surveyed publications, a ratio that increases up to 91% in those papers published since 2015. The Wilcoxon test, a.k.a. Mann-Whitney *U* test, is the most frequently applied test for pairwise comparison, whereas $\hat{A}_{12}$ by Vargha and Delaney is conducted for effect size measurement. This choice is in line with the recommendations given by Arcuri and Briand.

To the best of our knowledge, there is no specific guidelines detailing how many-objective algorithms should be evaluated. Most of the literature in many-objective optimisation considers the same

quality indicators proposed for multi-objective optimisation, and so happens in the analysed SBSE publications. More specifically, the most commonly applied indicators are hypervolume, inverted generational distance, PF size, spread and coverage. According to a practical guide (Wang et al., 2016), HV is not recommended for studies with more than six objectives because of its high computational cost. Our findings seem to reflect this trend, since just a few works solving problems with more than six objectives consider HV in their experimentation. Finally, only a recent paper (Xiang et al., 2018) reports values for a quality indicator specifically designed for many-objective optimisation, called pure diversity (Wang et al., 2017). The rest of indicators referred in Section 2.2 have not been considered in SBSE-related publications yet, probably because their use is not so widely extended than other indicators like hypervolume or IGD.

An interesting point about the empirical analysis refers to the graphic representation of the results. Visualisation of high-dimensional PFs is one of the key challenges of many-objective optimisation, and novel ways of representation are being explored (Walker et al., 2013). In the case of SBSE, the following techniques have been considered up to now: 2D or 3D scatter plots, showing some combinations of objectives or projecting the original PF; pairwise coordinate plots, actually a collection of all possible combinations of 2D scatter plots; parallel coordinates, each solution being represented as a line cutting vertical axes for each objective at a certain value; and radar plots, analogous to the previous one but representing objectives as vertices of a polygon.

Finally, reproducibility is an essential aspect in any experimental research field. A positive trend in SBSE papers consists in making publicly available replication packages, and the same applies to recent publications in the area related to many-objective optimisation. Even though only 34% of the analysed publications give access to algorithm implementations or benchmarks, this percentage increases up to 45% if only those works published after 2015 are considered. For those studies providing some additional material, the most frequently shared elements include code artefacts (23%) and problem instances (23%). Additional results are publicly available (13%) less often.

8.4. Practical implications

In addition to the aspects of the search proposal itself, there are other considerations affecting the software engineer that should be taken into account. One relevant point refers to the number of returned solutions, which is mentioned just by a few studies. It is desirable to offer the engineer a variety of alternatives, even though an excessive number of solutions could result in an information overload. Among the analysed studies, there are PFs comprised of hundreds or even thousands of solutions in domains like project scheduling (Kremmel et al., 2011) and service composition (Trummer et al., 2014). Some authors have proposed ways to reduce the number of alternatives based on establishing acceptance ranges (Bowman et al., 2010) or the use of decision-making techniques to assist the engineer to discriminate the best choices (Liu et al., 2007). Just the opposite situation could happen if there are not conflicting objectives, since it could lead to the possibility of returning one only solution (Assunção et al., 2014).

Another factor influencing practicability is related to time requirements. Some studies evaluate how execution time is affected by the increase of the number of objectives. While some authors mention that execution time does not experience a significant change (Assunção et al., 2014), others discuss a remarkable increase (Zheng et al., 2016). Other times, the computational cost can be attributed to the problem itself. For instance, modularisation and refactoring might require several hours to complete (Mkaouer et al., 2015; 2016). Also, scalability could depend on one partic-

ular characteristic of the problem. This is the case of the number of features and constraints in SPL (Sayyad et al., 2013a), or the number of services in service composition (Trummer et al., 2014). In fact, these factors could lead to prohibitive execution times when exhaustive search is considered (Wada et al., 2012; Olaechea et al., 2014). Some authors have studied the time required to reach acceptable results too, e.g. a minimum code coverage (Panichella et al., 2018) or a valid product configuration (Tan et al., 2015). Finally, the nature of the algorithm itself can be another determinant factor. It should be noted that novel many-objective evolutionary algorithms usually propose complex procedures that make them slower than previous multi-objective approaches, even when they will not necessarily produce better results (Ramírez et al., 2016).

9. Gaps and challenges

After an in-depth analysis of the state-of-the-art, we can identify gaps in at least four areas: (1) SE activities to be further explored; (2) dependencies between objectives; (3) advanced approaches to dynamically handle many objectives during search; and (4) exploration of industrial opportunities brought by ongoing research in many-objective optimisation. They all represent new challenges to be addressed in the future.

Exploration of SE activities. The underdevelopment of the project management and requirement phases with respect to the others is particularly noteworthy. In both cases, the variety of problems is more restricted, and the number of required objectives seems shorter. Recent studies are apparently moving towards more realistic formulations of the project scheduling problem, for which new objectives have been defined. Other planning tasks could experience a similar process. It is likely that decision criteria other than stakeholders' satisfaction, cost and effort could appear in requirements negotiation. For instance, potential risk, the expected market value or some organisational factors could be candidates for additional objectives. In addition, there are other application domains that deserve to be explored. As an example, those design activities requiring architectural decisions (Räihä, 2010) or cloud-oriented solutions (Harman et al., 2013) might require quantifying a large number of decision factors, probably as many as non-functional requirements. Related to automatic programming, genetic improvement looks for enhanced versions of existing software in terms of execution time, energy or memory consumption, among others (Petke et al., 2018). However, very few works have considered more than one non-functional property, even though improving one could greatly deteriorate others. The major challenge here is how to quantify these properties, since estimations might not be always available and simulations could be time-consuming.

Definition of objectives. We observe that the way in which objectives are currently defined and handled will bring important research opportunities. Actually, the lack of agreement on how software properties should be accurately measured or estimated is a historical issue in software engineering, which clearly affects the definition of realistic, quantitative objectives. Besides, objective functions are often highly dependent of the problem encoding. These aspects come from the intrinsic limitations of SBSE, where assumptions and simplifications of the real-world problem have to be done. Nevertheless, they become more evident under the many-objective perspective. A second concern refers to the real need of optimising a large number of objectives. As detailed in Sections 4 to 7, there is a significant number of problems for which many objectives have been considered. However, most studies do not analyse possible interdependences between them, nor compare the outcomes generated by different metric combinations. As the number of objectives become more relevant, some proof about the existence of conflicting objectives should be given to

justify the execution of many-objective algorithms. Furthermore, it would be also important to discuss in which ways these extra objectives contribute to the problem solution from the engineer's point of view. In this sense, considering that SE activities are often human-centred, engineers should be able to get more involved in both the problem definition and the search process. Only one study has adopted an interactive approach to solve a many-objective SE problem so far (Bavota et al., 2012). Even though other approaches have been already proposed to achieve human interaction, there is still a lot of research to be done (Ramírez et al., 2018).

Development of many-objective methods. The rapid growth of many-objective optimisation has led to the availability of an extensive catalogue of algorithms, making the selection a difficult decision. As discussed in Section 8.2, a further analysis on how the different families would satisfy the particular problem requirements could be valuable in SBSE. For instance, a software developer might not want a large number of solutions to be returned. Here, criteria others than quality indicators should be considered when discussing the strengths of a novel approach. In addition, notice that the findings of this survey clearly show a strong bias towards the use of evolutionary algorithms. Apart from the popularity of evolutionary techniques within SBSE, this could be explained by the fact that swarm-based metaheuristics have not been explored so widely in the context of many-objective optimisation. As other many-objective metaheuristics (Figueiredo et al., 2016) appear, it would be interesting to study their suitability for SBSE.

Industrial applicability of many-objective approaches. Finally, the gap between academy and industry is still a challenge in SBSE, becoming even more evident with the application of many-objective methods. Very recently, first industrial case studies are appearing in domains like modularisation (Mkaouer et al., 2015) and testing (Lachmann et al., 2017). Similarly, two studies in the context of refactoring (Mkaouer et al., 2016) and MDE (Fleck et al., 2017) have reported some experiences with industrial partners. These studies are essential in order to prove the performance of these search techniques in real contexts and reveal current limitations. Besides, the lack of tools is an important limiting factor. Only a few many-objective implementations for test case generation are now available as extensions of EvoSuite (Panichella et al., 2015; 2018).

10. Concluding remarks

The need of including many decision factors is encouraging SBSE researchers to explore the use of more advanced techniques. This survey has analysed 60 studies, reporting a great variety of SE tasks formulated as many-objective problems. Even though there is still a certain bias towards popular multi-objective algorithms like NSGA-II, a growing interest in more specialised techniques can be observed. Indeed, more than half of the reviewed studies adopted many-objective approaches of diverse nature, e.g. based on indicators or new forms of dominance. In contrast, evolutionary techniques clearly predominate, with only a few studies applying other metaheuristics or exact methods.

The design of novel many-objective proposals in areas like testing and the proliferation of comparative studies seem to indicate that many-objective optimisation is breaking new ground in SBSE. The findings of this survey contribute to the acquisition of more solid knowledge on many-objective optimisation principles and techniques. What still needs to be studied in more detail is how to adapt these approaches to the particular requirements of SBSE.

The analysis of the gaps and challenges in the area reveals several lines of future research, such as the need of studying dependencies between objectives, or the benefit that complementary techniques like dimensionality reduction and prioritisation methods could bring. From the observed evolution of what can be re-

ferred as many-objective SBSE, we assume that this emerging area will continue its development as we face more complex SE decision scenarios.

Acknowledgments

This work was supported by the Spanish Ministry of Economy and Competitiveness [project TIN2017-83445-P]; the Spanish Ministry of Education [grant FPU13/01466]; and FEDER funds.

References

- Aleti, A., Buhnova, B., Grunske, L., Koziol, A., Meedeniya, I., 2013. Software architecture optimization methods: a systematic literature review. *IEEE Trans. Softw. Eng.* 39 (5), 658–683.
- Amarjeet Chhabra, J. K., 2014a. An empirical study of the sensitivity of quality indicator for software module clustering. In: *Proceedings of the 7th International Conference Contemporary Computing (IC3)*. pp. 206–211.
- Amarjeet, Chhabra, J.K., 2014b. Preserving core components of object-oriented packages while maintaining structural quality. In: *Proceedings of the International Conference on Information and Communication Technologies (ICICT)*, pp. 833–840.
- Amarjeet, Chhabra, J.K., 2018. FP-ABC: Fuzzy-pareto dominance driven artificial bee colony algorithm for many-objective software module clustering. *Comput. Lang. Syst. Struct.* 51, 1–21.
- Arcuri, A., 2018a. Evomaster: evolutionary multi-context automated system test generation. In: *Proceedings of the IEEE 11th International Conference Software Testing, Verification and Validation (ICST)*, pp. 394–397. <https://github.com/EMResearch/EvoMaster>.
- Arcuri, A., 2018b. Test suite generation with the many independent objective (MIO) algorithm. *Inf. Softw. Technol.*
- Arcuri, A., Briand, L., 2014. A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering. *Softw. Test. Verif. Reliab.* 24 (8), 591–592.
- Asafuddoula, M., Ray, T., Sarker, R., 2013. A decomposition based evolutionary algorithm for many objective optimization with systematic sampling and adaptive epsilon control. In: *Evolutionary Multi-Criterion Optimization*. Springer Berlin Heidelberg, pp. 413–427.
- Asoudeh, N., Labiche, Y., 2013. A multi-objective genetic algorithm for generating test suites from extended finite state machines. In: *Proceedings of the 5th International Symposium on Search Based Software Engineering (SSBSE)*, pp. 288–293.
- Assunção, W.K.G., Colanzi, T.E., Vergilio, S.R., Pozo, A., 2013. On the application of the multi-evolutionary and coupling-based approach with different aspect-class integration testing strategies. In: *Proceedings of the 5th International Symposium on Search Based Software Engineering (SSBSE)*, pp. 19–33.
- Assunção, W.K.G., Colanzi, T.E., Vergilio, S.R., Pozo, A., 2014. A multi-objective optimization approach for the integration and test order problem. *Inf. Sci.* 267, 119–139.
- Bader, J., Zitzler, E., 2011. Hype: an algorithm for fast hypervolume-based many-objective optimization. *Evol. Comput.* 19 (1), 45–76.
- Bansiya, J., Davis, C.G., 2002. A hierarchical model for object-oriented design quality assessment. *IEEE Trans. Softw. Eng.* 28 (1), 4–17.
- Barros, M.D.O., 2012. An analysis of the effects of composite objectives in multi-objective software module clustering. In: *Proceedings of the 14th International Conference Genetic and Evolutionary Computation (GECCO)*, pp. 1205–1212.
- Bavota, G., Carnevale, F., De Lucia, A., Di Penta, M., Oliveto, R., 2012. Putting the Developer In-the-loop: An Interactive GA for Software Re-modularization. In: *Proceedings of the 4th International Symposium on Search Based Software Engineering*, pp. 75–89.
- Bavota, G., Di Penta, M., Oliveto, R., 2014. Evolving Software Systems. In: *Search Based Software Maintenance: Methods and Tools*. Springer Berlin Heidelberg, pp. 103–137. doi:10.1007/978-3-642-45398-4_4.
- Beume, N., Naujoks, B., Emmerich, M., 2007. SMS-EMOA: multiobjective selection based on dominated hypervolume. *Eur. J. Oper. Res.* 181 (3), 1653–1669.
- Boussaid, I., Lepagnot, J., Siarry, P., 2013. A survey on optimization metaheuristics. *Inf. Sci.* 237, 82–117.
- Boussaid, I., Siarry, P., Ahmed-Nacer, M., 2017. A survey on search-based model-driven engineering. *Automat. Softw. Eng.* 24 (2), 233–294.
- Bowman, M., Briand, L.C., Labiche, Y., 2010. Solving the class responsibility assignment problem in object-oriented analysis with multi-objective genetic algorithms. *IEEE Trans. Softw. Eng.* 36 (6), 817–837.
- Briand, L., Labiche, Y., Chen, K., 2013. A multi-objective genetic algorithm to rank state-based test cases. In: *Proceedings of the 5th International Symposium on Search Based Software Engineering*, pp. 66–80.
- Brockhoff, D., Zitzler, E., 2009. Objective reduction in evolutionary multiobjective optimization: theory and applications. *Evol. Comput.* 17 (2), 135–166.
- Campos, J., Ge, Y., Fraser, G., Eler, M., Arcuri, A., 2017. An empirical evaluation of evolutionary algorithms for test suite generation. In: *Proceedings of the 9th International Symposium Search Based Software Engineering (SSBSE)*, pp. 33–48.
- Canfora, G., Penta, M.D., Esposito, R., Villani, M.L., 2005. An approach for QoS-aware service composition based on genetic algorithms. In: *Proceedings of the 7th International Conference Genetic And Evolutionary Computation (GECCO)*, pp. 1069–1075.

- Chand, S., Wagner, M., 2015. Evolutionary many-objective optimization: a quick-start guide. *Surv. Operat. Res. Manag. Sci.* 20 (2), 35–42.
- Chen, J., Nair, V., Krishna, R., Menzies, T., 2018. "Sampling" as a baseline optimizer for search-based software engineering. *IEEE Trans. Softw. Eng.*
- Cheng, R., Jin, Y., Olhofer, M., Sendhoff, B., 2016. A reference vector guided evolutionary algorithm for many-objective optimization. In press.
- Chidamber, S.R., Kemerer, C.F., 1994. A metrics suite for object oriented design. *IEEE Trans. Softw. Eng.* 20 (6), 476–493.
- Coello Coello, C.A., Lamont, G.B., Van Veldhuizen, D.A., 2007. *Evolutionary algorithms for solving multi-Objective problems*, 2nd ed. Springer.
- Colanzi, T. E., Vergilio, S. R., 2012. Applying search based optimization to software product line architectures: Lessons learned. *Proceedings of the 4th International Symposium on Search Based Software Engineering (SSBSE)*, 259–266.
- De Campos Jr., A., Pozo, A.T.R., Vergilio, S.R., Savegnago, T., 2010. Many-objective Evolutionary Algorithms in the Composition of Web Services. In: *Proceedings of the 11th Brazilian Symposium on Neural Networks*, pp. 152–157.
- Dea, T.J., 2016. Improving the performance of many-objective software refactoring technique using dimensionality reduction. In: *Proceedings of the 8th International Symposium on Search Based Software Engineering (SSBSE)*, pp. 298–303.
- Deb, K., Jain, H., 2014. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part i: solving problems with box constraints. *IEEE Trans. Evol. Comput.* 18 (4), 577–601.
- Deb, K., Sundar, J., 2006. Reference point based multi-objective optimization using evolutionary algorithms. In: *Proceedings of the 8th Annual Conference Genetic and Evolutionary Computation (GECCO)*, pp. 635–642.
- Durillo, J.J., Nebro, A.J., 2011. jMetal: a java framework for multi-objective optimization. *Adv. Eng. Softw.* 42 (10), 760–771. <http://jmetal.github.io/jMetal/>.
- Farina, M., Amato, P., 2002. On the optimal solution definition for many-criteria optimization problems. In: *Proceedings of the Annual Meeting of the North American Fuzzy Information Processing Society*, pp. 233–238.
- Ferrucci, F., Harman, M., Sarro, F., 2014. Software project management in a changing world. In: *Search-Based Software Project Management*. Springer Berlin Heidelberg, pp. 373–399.
- Figueiredo, E., Ludermir, T., Bastos-Filho, C., 2016. Many objective particle swarm optimization. *Inf. Sci.* 374, 115–134.
- Finkelstein, A., Harman, M., Mansouri, S.A., Ren, J., Zhang, Y., 2009. A search based approach to fairness analysis in requirement assignments to aid negotiation. *Med. Decis. Making. Requir. Eng.* 14 (4), 231–245.
- Fleck, M., Troya, J., Kessentini, M., Wimmer, M., 2017. Model transformation modularization as a many-objective optimization problem. *IEEE T. Softw. Eng.* 43 (11), 1009–1032.
- Fleming, P.J., Purshouse, R.C., Lygoe, R.J., 2005. Many-objective Optimization: an engineering design perspective. In: *Evolutionary Multi-Criterion Optimization*. Springer Berlin Heidelberg, pp. 14–32.
- Fortin, F.A., De Rainville, F.M., Gardner, M.A.G., Parizéau, M., Gagné, C., 2012. DEAP: evolutionary algorithms made easy. *J. Mach. Learn. Res.* 13 (1), 2171–2175. <http://github.com/DEAP/deap>.
- Fraser, G., Arcuri, A., 2011. Evosuite: automatic test suite generation for object-oriented software. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, pp. 416–419. <http://www.evosuite.org>.
- Gómez, R.H., Coello, C.A.C., 2013. MOMBI: a new metaheuristic for many-objective optimization based on the R2 indicator. In: *Proceedings of the IEEE Congress on Evolutionary Computation (CEC)*, pp. 2488–2495.
- Guizzo, G., Vergilio, S.R., Pozo, A.T., Fritsche, G.M., 2017. A multi-objective and evolutionary hyper-heuristic applied to the integration and test order problem. *Appl. Soft Comput.* 56, 331–344.
- Guo, J., Liang, J.H., Shi, K., Yang, D., Zhang, J., Czarnecki, K., Ganesh, V., Yu, H., 2017. SMTBIEA: a hybrid multi-objective optimization algorithm for configuring large constrained software product lines. *Softw. Syst. Model.* 1–20.
- Harman, M., Afshin Mansouri, S., Zhang, Y., 2012. Search based software engineering: trends, techniques and applications. *ACM Comput. Surv.* 45 (1), 1–64.
- Harman, M., Jia, Y., Zhang, Y., 2015. Achievements, open problems and challenges for search based software testing. In: *Proceedings of the IEEE 8th International Conference Software Testing, Verification and Validation (ICST)*, pp. 1–12.
- Harman, M., Jones, B.F., 2001. Search-based software engineering. *Inf. Softw. Technol.* 43 (14), 833–839.
- Harman, M., Lakhotia, K., Singer, J., White, D.R., Yoo, S., 2013. Cloud engineering is search based software engineering too. *J. Syst. Softw.* 86 (9), 2225–2241.
- Henard, C., Papadakis, M., Harman, M., Le Traon, Y., 2015. Combining multi-objective search and constraint solving for configuring large software product lines. In: *Proceedings of the IEEE/ACM 37th International Conference Software Engineering (ICSE)*, pp. 517–528.
- Hernández Gómez, R., Coello Coello, C.A., 2015. Improved metaheuristic based on the r2 indicator for many-objective optimization. In: *Proceedings of the 17th Annual Conference on Genetic and Evolutionary Computation (GECCO)*. ACM, pp. 679–686.
- Hierons, R.M., Li, M., Liu, X., Segura, S., Zheng, W., 2016. SIP: Optimal product selection from feature models using many-objective evolutionary optimization. *ACM Trans. Softw. Eng. Method* 25 (2), 17:1–17:39.
- Hrubá, V., Krěna, B., Letko, Z., Pluháčková, H., Vojnar, T., 2014. Multi-objective genetic optimization for noise-based testing of concurrent software. In: *Proceedings of the 6th International Symposium on Search Based Software Engineering (SSBSE)*, pp. 107–122.
- Hughes, E., 2003. Multiple single objective pareto sampling. In: *Proceedings of the Congress on Evolutionary Computation (CEC)*, Vol. 4, pp. 2678–2684.
- Ishibuchi, H., Tsukamoto, N., Nojima, Y., 2008. Evolutionary many-objective optimization: a short review. In: *Proceedings of the IEEE Congress on Evolutionary Computation (CEC)*, pp. 2419–2426.
- Kalbousi, S., Bechikh, S., Kessentini, M., Ben Said, L., 2013. Preference-based many-objective evolutionary testing generates harder test cases for autonomous agents. In: *Proceedings of the 5th International Symposium on Search Based Software Engineering (SSBSE)*, pp. 245–250.
- Khalil, E., Assaf, M., Sayyad, A.S., 2017. Human Resource Optimization for Bug Fixing: Balancing Short-term and Long-term Objectives. In: *Proceedings of the 9th International Symposium on Search Based Software Engineering (SSBSE)*. Vol. 10452, pp. 124–129.
- Khoshgoftaar, T.M., Liu, Y., Seliya, N., 2004. A multiobjective module-order model for software quality enhancement. *IEEE Trans. Evolut. Comput.* 8 (6), 593–608.
- Khoshnevis, S., Shams, F., 2017. Automating identification of services and their variability for product lines using NSGA-II. *Front. Comput. Sci.* 11 (3), 444–464.
- Kitchenham, B., Charters, S., 2007. *Guidelines for performing systematic literature reviews in software engineering*. Keele University and Durham University Technical Report.
- Krall, J., Menzies, T., Davies, M., 2015. GALE: geometric active learning for search-based software engineering. *IEEE Trans. Softw. Eng.* 41 (10), 1001–1018.
- Kremmel, T., Kubalik, J., Biffl, S., 2011. Software project portfolio optimization with advanced multiobjective evolutionary algorithms. *Appl. Softw. Comput.* 11 (1), 1416–1426.
- Kumari, A.C., Srinivas, K., 2016. Hyper-heuristic approach for multi-objective software module clustering. *J. Syst. Softw.* 117, 384–401.
- Lachmann, R., Felderer, M., Niek, M., Schulze, S., Seidl, C., Schaefer, I., 2017. Multi-objective Black-box Test Case Selection for System Testing. In: *Proceedings of the 19th Genetic and Evolutionary Computation Conference (GECCO)*, pp. 1311–1318.
- Laumanns, M., Thiele, L., Deb, K., Zitzler, E., 2002. Combining convergence and diversity in evolutionary multiobjective optimization. *Evol. Comput.* 10 (3), 263–282.
- Li, M., Yang, S., Liu, X., Shen, R., 2013. A Comparative Study on Evolutionary Algorithms for Many-Objective Optimization. In: *Evolutionary Multi-Criterion Optimization*, pp. 261–275.
- Li, B., Li, J., Tang, K., Yao, X., 2015. Many-objective evolutionary algorithms. *ACM Comput. Surv.* 48 (1), 1–35.
- Li, K., Deb, K., Zhang, Q., Kwong, S., 2015. An evolutionary many-objective optimization algorithm based on dominance and decomposition. *IEEE Trans. Evol. Comput.* 19 (5), 694–716.
- Li, M., 2015. *Evolutionary many-objective optimisation: pushing the boundaries*. Brunel University London Ph.D. thesis.
- Li, M., Chen, T., Yao, X., 2018a. A critical review of: "a practical guide to select quality indicators for assessing pareto-based search algorithms in search-based software engineering": essay on quality indicator selection for sbse. In: *Proceedings of the 40th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER '18)*, pp. 17–20.
- Li, K., Wang, R., Zhang, T., Ishibuchi, H., 2018b. Evolutionary many-objective optimization: A comparative study of the state-of-the-art. *IEEE Access* 6, 26194–26214.
- Li, M., Grosan, C., Yang, S., Liu, X., Yao, X., 2018c. Multiline distance minimization: A visualized many-objective test problem suite. *IEEE T. Evolut. Comput.* 22 (1), 61–78.
- Li, M., Yang, S., Liu, X., 2014. Shift-based density estimation for pareto-based algorithms in many-objective optimization. *IEEE Trans. Evol. Comput.* 18 (3), 348–365.
- Li, M., Yang, S., Liu, X., 2015. A performance comparison indicator for pareto front approximations in many-objective optimization. In: *Proceedings of the Annual Conference Genetic and Evolutionary Computation (GECCO'15)*. GECCO '15. ACM, New York, NY, USA, pp. 703–710. <http://doi.acm.org/10.1145/2739480.2754687>.
- Liu, Z., Guo, H.Q., Li, D., Tao, H., Zhang, J.J., 2007. Solving multi-objective and fuzzy multi-attribute integrated technique for qos-aware web service selection. In: *Proceedings of the International Conference Wireless Communications, Networking and Mobile Computing (WiCOM)*, pp. 735–739.
- Lopez-Herrejon, R.E., Linsbauer, L., Egyed, A., 2015. A systematic mapping study of search-based software engineering for software product lines. *Inf. Softw. Technol.* 61, 33–51.
- López Jaimes, A., Coello Coello, C.A., 2015. *Springer handbook of computational intelligence*. In: *Many-Objective Problems: Challenges and Methods*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 1033–1046.
- Luke, S., 2017. ECJ Then and Now. In: *Proceedings of the Companion Publication 2017 Annual Genetic and Evolutionary Computation Conference GECCO Companion '17*. ACM, pp. 1223–1230. <http://cs.gmu.edu/~eclab/projects/ecj/>.
- Mariani, T., Vergilio, S.R., 2017. A systematic review on search-based refactoring. *Inf. Softw. Technol.* 83, 14–34.
- MathWorks, 2018. *Matlab Optimization Toolbox*. <https://www.mathworks.com/products/optimization.html>.
- McMinn, P., 2011. Search-based software testing: past, present and future. In: *Proceedings of the IEEE 4th International Conference Software Testing, Verification and Validation Workshops*, pp. 153–163.
- Mkaouer, M.W., Kessentini, M., Bechikh, S., Deb, K., Ó Cinnéide, M., 2014. High dimensional search-based software engineering: finding tradeoffs a Mong 15 objectives for automating software refactoring using NSGA-III. In: *Proceedings of the 16th International Conference Genetic and Evolutionary Computation (GECCO)*, pp. 1263–1270.
- Mkaouer, M.W., Kessentini, M., Bechikh, S., Ó Cinnéide, M., Deb, K., 2016. On the use of many quality attributes for software refactoring: a many-objective

- live search-based software engineering approach. *Empir. Softw. Eng.* 21 (6), 2503–2545.
- Mkaouer, W., Kessentini, M., Shaout, A., Kontchou, P., Deb, K., Bechikh, S., Ouni, A., 2015. Many-objective software remodularization using NSGA-III. *ACM Trans. Softw. Eng. Method* 24 (3).
- Nebro, A., Durillo, J., Garcia-Nieto, J., Coello Coello, C., Luna, F., Alba, E., 2009. SMPSO: a new PSO-based metaheuristic for multi-objective optimization. In: *IEEE Symposium on Computational Intelligence in Multi-Criteria Decision-Making (MCDM)*, pp. 66–73.
- Olaechea, R., Rayside, D., Guo, J., Czarnecki, K., 2014. Comparison of exact and approximate multi-objective optimization for software product lines. In: *Proceedings of the 18th International Software Product Line Conference (SPLC)*, 1, pp. 92–101.
- Palomba, F., Panichella, A., Zaidman, A., Oliveto, R., De Lucia, A., 2016. Automatic Test Case Generation: What If Test Code Quality Matters? In: *Proceedings of the 25th International Symposium Software Testing and Analysis (ISSTA)*, pp. 130–141.
- Panichella, A., Kifetew, F., Tonella, P., 2018. Automated test case generation as a many-objective optimisation problem with dynamic selection of the targets. *IEEE Trans. Softw. Eng.* 44 (2), 122–158.
- Panichella, A., Kifetew, F.M., Tonella, P., 2015. Reformulating branch coverage as a many-objective optimization problem. In: *Proceedings of the IEEE 8th International Conference Software Testing, Verification and Validation (ICST)*, pp. 1–10.
- Panichella, A., Kifetew, F.M., Tonella, P., 2017. LIPS vs MOSA: a replicated empirical study on automated test case generation. In: *Proceedings of the 9th International Symposium Search Based Software Engineering (SSBSE)*, pp. 83–98.
- Petke, J., Haraldsson, S.O., Harman, M., Langdon, W.B., White, D.R., Woodward, J.R., 2018. Genetic improvement of software: a comprehensive survey. *IEEE T. Evolut. Comput.* 22 (3), 415–432.
- Pitangueira, A.M., Maciel, R.S.P., Barros, M., 2015. Software requirements selection and prioritization using SBSE approaches: a systematic review and mapping of the literature. *J. Syst. Softw.* 103, 267–280.
- Praditwong, K., Harman, M., Yao, X., 2011. Software module clustering as a multi-objective search problem. *IEEE Trans. Softw. Eng.* 37 (2), 264–282.
- Purshouse, R.C., Fleming, P.J., 2003. Evolutionary many-objective optimisation: an exploratory analysis. In: *Proceedings of the IEEE Congress on Evolutionary Computation (CEC)*, pp. 2066–2073.
- Räihä, O., 2010. A survey on search-based software design. *Comput. Sci. Rev.* 4 (4), 203–249.
- Ramírez, A., Parejo, J.A., Romero, J.R., Segura, S., Ruiz-Cortés, A., 2017. Evolutionary composition of QoS-aware web services: a many-objective perspective. *Expert Syst. Appl.* 72, 357–370.
- Ramírez, A., Romero, J.R., Simons, C., 2018. A systematic review of interaction in search-based software engineering. *IEEE Trans. Softw. Eng.*
- Ramírez, A., Romero, J.R., Ventura, S., 2016. A comparative study of many-objective evolutionary algorithms for the discovery of software architectures. *Empir. Softw. Eng.* 21 (6), 2546–2600.
- Rodríguez, D., Ruiz, M., Riquelme, J.C., Harrison, R., 2011. Multiobjective simulation optimisation in software project management. In: *Proceedings of the 13th International Conference Genetic and Evolutionary Computation (GECCO)*, pp. 1883–1890.
- Saber, T., Brevet, D., Botterweck, G., Ventresque, A., 2018. Is seeding a good strategy in multi-objective feature selection when feature models evolve? *Inf. Softw. Technol.* 95, 266–280.
- Sarro, F., Ferrucci, F., Gravino, C., 2012. Single and multi objective genetic programming for software development effort estimation. In: *Proceedings of the 27th Annual ACM Symposium on Applied Computing (SAC)*, 1221–1226.
- Sayyad, A. S., Ammar, H., 2013. Pareto-optimal search-based software engineering (POSBSE): a literature survey. In: *Proceedings of the 2nd International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering (RAISE)*, 21–27.
- Sayyad, A.S., Ingram, J., Menzies, T., Ammar, H., 2013. Scalable product line configuration: a straw to break the Camel's back. In: *Proceedings of the 28th IEEE/ACM International Conference Automated Software Engineering (ASE)*, pp. 465–474.
- Sayyad, A.S., Menzies, T., Ammar, H., 2013. On the value of user preferences in search-based software engineering: a case study in software product lines. In: *Proceedings of the 35th International Conference Software Engineering (ICSE)*, pp. 492–501.
- Shen, X., Minku, L.L., Bahsoon, R., Yao, X., 2016. Dynamic software project scheduling through a proactive-rescheduling method. *IEEE Trans. Softw. Eng.* 42 (7), 658–686.
- Shen, X.N., Minku, L.L., Marturi, N., Guo, Y.N., Han, Y., 2018. A q-learning-based memetic algorithm for multi-objective dynamic software project scheduling. *Inf. Sci.* 428, 1–29.
- Sierra, M.R., Coello Coello, C.A., 2005. Improving PSO-based multi-objective optimization using crowding, mutation and ϵ -dominance. In: *Proceedings of the Evolutionary Multi-Criterion Optimization*, 3410, pp. 505–519. *Lecture Notes in Computer Science*
- Singh, H.K., Isaacs, A., Ray, T., 2011. A pareto corner search evolutionary algorithm and dimensionality reduction in many-objective optimization problems. *IEEE Trans. Evol. Comput.* 15 (4), 539–556.
- Tan, T., Xue, Y., Chen, M., Sun, J., Liu, Y., Dong, J., 2015. Optimizing selection of competing features via feedback-directed evolutionary algorithms. In: *Proceedings International Symp. Software Testing and Analysis (ISSTA)*, pp. 246–256.
- Thiele, L., Miettinen, K., Korhonen, P.J., Molina, J., 2009. A preference-based evolutionary algorithm for multi-objective optimization. *Evol. Comput.* 17 (3), 411–436.
- Trummer, I., Faltings, B., Binder, W., 2014. Multi-objective quality-driven service selection – a fully polynomial time approximation scheme. *IEEE T. Softw. Eng.* 40 (2), 167–191.
- Ventura, S., Romero, C., Zafra, A., Delgado, J.A., Hervás, C., 2008. JCLEC: a java framework for evolutionary computation. *Soft Comput.* 12 (4), 381–392. <http://jclec.sourceforge.net/>
- Wada, H., Suzuki, J., Yamano, Y., Oba, K., 2012. E3: a multiobjective optimization framework for SLA-aware service composition. *IEEE Trans. Serv. Comput.* 5 (3), 358–372.
- Walker, D.J., Everson, R.M., Fieldsend, J.E., 2013. Visualizing mutually nondominating solution sets in many-objective optimization. *IEEE Trans. Evolut. Comput.* 17 (2), 165–184.
- Wang, H., Jiao, L., Yao, X., 2015. Two_arch2: an improved two-archive algorithm for many-objective optimization. *IEEE Trans. Evol. Comput.* 19 (4), 524–541.
- Wang, H., Jin, Y., Yao, X., 2017. Diversity assessment in many-objective optimization. *IEEE Trans. Cybern.* 47 (6), 1510–1522.
- Wang, S., Ali, S., Yue, T., Li, Y., Liaaen, M., 2016. A practical guide to select quality indicators for assessing pareto-based search algorithms in search-based software engineering. In: *Proceedings of the IEEE/ACM 38th International Conference Software Engineering (ICSE)*, pp. 631–642.
- Wei, Z., Xiaoxue, W., Yang, X., Shichao, C., Wenxin, L., Jun, L., 2017. Test suite minimization with mutation testing-based many-objective evolutionary optimization. In: *Proceedings of the International Conference Software Analysis, Testing and Evolution (SATE)*, pp. 30–36.
- Wilkerson, J.L., Tauritz, D., 2012. Multi-objective coevolutionary automated software correction. In: *Proceedings of the 14th International Conference Genetic and Evolutionary Computation (GECCO)*, pp. 1391–1392.
- Xiang, Y., Zhou, Y., Li, M., Chen, Z., 2017. A vector angle-based evolutionary algorithm for unconstrained many-objective optimization. *IEEE Trans. Evol. Comput.* 21 (1), 131–152.
- Xiang, Y.L., Zhou, Y., Zheng, Z., Li, M., 2018. Configuring software product lines by combining many-objective optimization and SAT solvers. *ACM Trans. Softw. Eng. Methodol.* 26 (4), 14:1–14:46.
- Xue, Y., Zhong, J., Tan, T.H., Liu, Y., Cai, W., Chen, M., Sun, J., 2016. IBED: combining IBEA and DE for optimal feature selection in software product line engineering. *Appl. Soft Comput.* 49, 1215–1231.
- Yang, S., Li, M., Liu, X., Zheng, J., 2013. A grid-based evolutionary algorithm for many-objective optimization. *IEEE Trans. Evol. Comput.* 17 (5), 721–736.
- Yu, Y., Ma, H., Zhang, M., 2015. F-MOGP: a novel many-objective evolutionary approach to QoS-aware data intensive web service composition. In: *Proceedings of the IEEE Congress on Evolutionary Computation (CEC)*, pp. 2843–2850.
- Zhang, Q., Li, H., 2007. MOEA/D: a multiobjective evolutionary algorithm based on decomposition. *IEEE Trans. Evol. Comput.* 11 (6), 712–731.
- Zhang, Y., Harman, M., Finkelstein, A., Afshin Mansouri, S., 2011. Comparing the performance of metaheuristics for the analysis of multi-stakeholder tradeoffs in requirements optimisation. *Inf. Softw. Technol.* 53 (7), 761–773.
- Zheng, W., Hierons, R.M., Li, M., Liu, X.H., Vinciotti, V., 2016. Multi-objective optimisation for regression testing. *Inf. Sci.* 334–335, 1–16.
- Zhou, A., Qu, B.Y., Li, H., Zhao, S.Z., Suganthan, P.N., Zhang, Q., 2011. Multiobjective evolutionary algorithms: a survey of the state of the art. *Swarm Evol. Comput.* 1 (1), 32–49.
- Zitzler, E., Künzli, S., 2004. Indicator-based Selection in multiobjective search. In: *Proceedings of the Parallel Problem Solving from Nature (PPSN VIII)*, 3242, pp. 832–842. *Lecture Notes in Computer Science*
- Zou, X., Chen, Y., Liu, M., Kang, L., 2008. A new evolutionary algorithm for solving many-objective optimization problems, Vol. 38, pp. 1402–1412.

Aurora Ramírez received the M.Sc. and Ph.D. degrees in Computer Science from the University of Córdoba, Córdoba, Spain, in 2012 and 2018, respectively. She is a member of the Knowledge Discovery and Intelligent Systems Research Laboratory, where she has conducted her research in search-based software design and evolutionary computation. Her research interests include search-based software engineering and metaheuristics, with particular focus on multi-objective and interactive optimisation.

José Raúl Romero received the M.Sc. and Ph.D. degrees in computer science from the University of Málaga, Spain, in 2003 and 2007, respectively. He is currently an Associate Professor with the Department of Computer Science and Numerical Analysis, University of Córdoba. He has published over 100 papers in journals, books and scientific conferences, and collaborated in different research projects and excellence networks. His research interests include the development of intelligent systems for supporting the software development process and the democratization and automation of data science. He can be reached at <http://www.jromero.net/en>.

Sebastián Ventura received the B.Sc. and Ph.D. degrees in sciences from the University of Córdoba, Córdoba, Spain, in 1989 and 1996, respectively. He is currently a Full Professor with the Department of Computer Science and Numerical Analysis, University of Córdoba, where he heads the Knowledge Discovery and Intelligent Systems Research Laboratory. He has published over 250 papers in journals and scientific conferences, and he has edited three books and several special issues in international journals. He has also been engaged in 12 research projects. His current research interests include soft-computing, machine learning, and data mining and their applications.